

Understanding the API Dump (api_dump_v2.json)

The provided JSON dump is a log of GraphQL responses from PoolPlayers' API. As with any GraphQL API, responses come back as JSON objects whose fields mirror exactly what was requested ¹ ². Each response has a top-level `data` key, under which the returned fields appear ². In practice, this means that when the client asks for specific stats fields (e.g. `ninesPlayed` or `matchesWon`), those same field names will appear in the JSON response. By design GraphQL responses are formatted as JSON ³, so all numeric stats and relationships show up as JSON properties.

Member and Alias Stats

The JSON contains data from several key GraphQL queries related to players' stats. For example:

- **Member Summary** (`getMemberStatsHeader`): This query (by member ID) returns overall counts of games played. In the JSON under `data.member` you'll see fields like `ninesPlayed` and `eightsPlayed` that indicate how many 9-ball or 8-ball games the member has played ². It also lists the member's aliases (one per league or name variation) and high-level stats (e.g. skill level, CLA).
- **Alias Stats** (`getEightBallStats`): Using an alias ID (one of the member's league aliases), this query returns detailed stats in a nested structure. In the JSON under `data.alias`, you'll find two stat objects: `EightBallStats` and `NineBallStats` (each typically an array of lifetime statistics objects). For each of those, fields like `matchesPlayed`, `matchesWon`, `CLA`, `defensiveShotAvg`, etc., appear with their numeric values. For example, Justin Ross's alias might show `EightBallStats[0].matchesPlayed = 28` and `NineBallStats[0].matchesPlayed = 32`. In addition, there is a `players` array under the alias. Each element of `players` has a `__typename` of either `"EightBallPlayer"` or `"NineBallPlayer"`, reflecting the format of that entry ¹. Within each player object you see format-specific fields: for an **EightBallPlayer**, fields like `matchesPlayed`, `matchesWon`, `eightOnBreaks`, etc.; for a **NineBallPlayer**, fields like `matchesPlayed`, `matchesWon`, `nineOnSnaps`, etc. These JSON entries exactly match the fragments requested in the query.
- **Session Stats** (`AliasSessionStats`): This query (and its dropdown variant) fetches stats for one specific session (year). It returns similar fields as above but scoped to a given session and format. For example, it can return how many break-and-runs a player had in 8-ball vs 9-ball during that season. The JSON structure includes the session ID and then the same kinds of fields (`eightBallBreakAndRuns`, `nineBallBreakAndRuns`, `matchesPlayed`, etc.) under the `alias.players` objects.

Teams, Divisions, and Other Data

Other parts of the dump show team and division information in JSON form:

- **Teams and Rosters:** Queries like `teamRoster`, `teamPage`, or `divisionRosters` return lists of players on a team. In those JSON responses you'll see arrays of player objects (with fields like `id`, `displayName`, `memberNumber`, etc.) and may see their stats or season data. For example, `divisionStandings` or `divisionContent` might return a list of teams (`teams` array) each with `id`, `name`, `teamNumber`, and related stats like matches won/lost. All of these again appear as JSON arrays under the corresponding keys, following the GraphQL query shape ¹.
- **League and Division Info:** Queries such as `leagueInfo`, `leagueLayout`, and `divisionContent` provide structural details. The JSON for `leagueInfo` might list the league's ID, name, and current session, while `divisionContent` will include each division's ID and an array of teams. All requested fields (division names, numbers, etc.) appear as JSON properties in the response.
- **Membership History (`getMembershipHistory`):** For a given alias, this query returns the history of that member across seasons. The JSON includes a `membershipHistory` list of objects with fields like `year` and `leaguePaidIn`, showing which years and leagues the member was active.
- **Events/Tournaments:** Queries like `leagueEvents` or `getTournament` return arrays of event objects (tournament details, dates, location, etc.). These appear as JSON lists under keys such as `data.league.events` or `data.tournament`, with fields exactly matching those requested in the GraphQL query.

Across all these sections, the pattern is the same: each GraphQL query's response is a JSON object where the keys and nested structure (arrays, objects) correspond exactly to the queries made. For instance, if you query an alias for players, the JSON will contain an array named `players`; if you query division standings, the JSON will contain a list of teams with their win/loss records. This mirroring of the query structure is a fundamental feature of GraphQL responses ¹.

Key takeaways: GraphQL responses are JSON, so numeric and relational data (player stats, team records, etc.) show up as JSON fields and nested objects. Player stats for 9-ball and 8-ball are explicitly provided in fields like `matchesPlayed`, `matchesWon`, and counts (e.g. `ninesPlayed`, `eightsPlayed`), and these are found under either the member-level or alias-level results as shown above. Each data category (member, alias, division, team, event) appears under its own key and follows the exact structure of the query, making it possible to trace, for example, where Justin Ross's 9-ball stats are recorded (in `data.alias.NineBallStats` for the alias, or in `data.alias.players` entries where `__typename` is `"NineBallPlayer"`). The JSON dump thus contains everything needed to map players to their stats in each format and session.

Sources: The response format and field naming conventions follow the GraphQL specification and typical API design ¹ ², which dictate that responses are JSON and mirror the requested query fields.

